Algorithms Used in the APL Smart Encryption Advisor & Engine

A Complete Reference — What, Why, Where, and How

Table of Contents

1. Overview — How Algorithms Are Selected
2. Supporting Algorithm: Shannon Entropy
3. Supporting Algorithm: PBKDF2-HMAC-SHA256 (Key Derivation)
4. Encryption Algorithm 1: AES-256-GCM
5. Encryption Algorithm 2: ChaCha20-Poly1305
6. Encryption Algorithm 3: Multi-byte XOR
7. Encryption Algorithm 4: ROT Cipher (Caesar Cipher)
8. Algorithm Comparison Table

1. Overview — How Algorithms Are Selected

This project does not apply one fixed algorithm to every file. Instead, it **measures the data first**, then **chooses the best algorithm** based on those measurements.

The selection pipeline has two phases:

```
Phase 1 — Measure (profiler.py)
──────────────────────────────────

  Input file / text
       │
       ├─ Shannon Entropy    →  how random / unpredictable are the bytes?
       ├─ Printable Ratio    →  what fraction of bytes are readable ASCII?
       └─ Byte Size          →  how large is the data?

Phase 2 — Match (selector.py)
──────────────────────────────────

  Measurements
       │
       ├─ High printable (≥90%) + Low entropy (<3.5)   →  ROT Cipher
       ├─ High printable (≥85%) + Moderate entropy      →  AES-256-GCM
       ├─ Very high entropy (>7.0)                      →  ChaCha20-Poly1305
       └─ Everything else                               →  ChaCha20-Poly1305
```

Every encryption algorithm listed below is implemented in **`crypto_engine.py`**. The selection logic lives in **`selector.py`**. The measurements are computed in **`profiler.py`**.

2. Supporting Algorithm: Shannon Entropy

What It Is

Shannon Entropy is a mathematical formula from **information theory**, invented by Claude Shannon in 1948. It measures the **average unpredictability** of a byte in the input data. The result is expressed as **bits per byte**, ranging from **0.0** (perfectly predictable) to **8.0** (perfectly random).

The Formula

$$H = -\sum_{x} p(x) \cdot \log_2(p(x))$$

Where:

- `x` is each possible byte value (0 through 255)
- `p(x)` is the probability of that byte value appearing in the data
- `log₂` is the logarithm in base 2
- `Σ` means "sum over all values of x"

- The minus sign makes the result positive ($\log_2$ of a fraction is negative)

Typical Entropy Values

Data Type	Entropy Range	Example
All identical bytes (`AAAAAA…`)	0.0	Repeated characters
Simple plain English text	3.5 – 4.5	A story or email
Structured data (CSV, JSON)	4.5 – 6.5	Database exports
Compressed files (.zip, .gz)	7.0 – 7.9	Zip archives
Encrypted data	7.5 – 8.0	AES ciphertext
Truly random bytes	≈ 8.0	`os.urandom()` output

Where It Is Implemented

File: — function `shannon_entropy(data: bytes) -> float`

How It Is Implemented (Step by Step)

```
python
def shannon_entropy(data: bytes) -> float:
    if not data:
        return 0.0                          # empty input has no entropy

    # Step 1: Count how many times each byte value (0–255) appears
    freq: dict[int, int] = {}
    for byte in data:
        freq[byte] = freq.get(byte, 0) + 1

    # Step 2: Apply the Shannon formula
    total   = len(data)
    entropy = 0.0
    for count in freq.values():
        probability = count / total         # p(x)
        entropy    -= probability * math.log2(probability)  # −p·log₂(p)

    return round(entropy, 4)
```

Concrete example with the word "HELLO":

Byte	Value	Count	p(x)	$p(x) \cdot \log_2(p(x))$
H	72	1	1/5 = 0.2	0.2 · (−2.322) = −0.464
E	69	1	1/5 = 0.2	0.2 · (−2.322) = −0.464
L	76	2	2/5 = 0.4	0.4 · (−1.322) = −0.529
O	79	1	1/5 = 0.2	0.2 · (−2.322) = −0.464

H = −(−0.464 − 0.464 − 0.529 − 0.529 − 0.464) = **1.921 bits/byte**

What Data It Operates On

- **Input:** Any sequence of raw bytes — text files, binary files, CSV data, images, or a typed string
- **Output:** A single floating-point number (the entropy score)

Requirements for Implementation

Requirement	Detail
Python standard library	`math` module (for `math.log2`)
Input type	`bytes` object
Special handling	Guard against empty input (division by zero)
No third-party library needed	Pure Python arithmetic

3. Supporting Algorithm: PBKDF2-HMAC-SHA256 (Key Derivation)

What It Is

PBKDF2 stands for **Password-Based Key Derivation Function 2**. It is defined in **RFC 8018** and is used to convert a human-readable passphrase (like `"MySecret123!"`) into a fixed-length cryptographic key (32 bytes for AES/ChaCha20).

It is NOT an encryption algorithm — it is a **key stretching** function that sits between the user's input and the actual cipher.

Why It Is Needed

A passphrase typed by a user has two problems as a direct encryption key:

1. **Wrong length —** AES needs exactly 32 bytes; passphrases vary wildly
2. **Not random —** short or common passphrases are easily guessed by attackers

PBKDF2 solves both problems by iterating a hash function thousands of times.

The Algorithm

```
Step 1:  Encode passphrase to UTF-8 bytes
Step 2:  Generate a random 16-byte salt (stored in the .enc file)
Step 3:  Compute:
           T₁ = HMAC-SHA256(passphrase, salt || 0x00000001)
           T₂ = HMAC-SHA256(passphrase, T₁)
           T₃ = HMAC-SHA256(passphrase, T₂)
           ...  (repeat 600,000 times)
           Tₙ = HMAC-SHA256(passphrase, Tₙ₋₁)
Step 4:  Output = Tₙ  (32 bytes, used as the cipher key)
```

Each iteration takes the output of the previous one as input. After 600,000 rounds, the output is cryptographically indistinguishable from a truly random 32-byte sequence.

The Role of the Salt

The salt is a **16 random bytes** generated fresh for every encryption (`os.urandom(16)`). It is stored unencrypted in the `.enc` file header.

The salt ensures that even if two users encrypt with the **exact same passphrase**, they get **completely different keys —** because their salts differ. This defeats pre-computed "rainbow table" attacks.

Where It Is Implemented

File: — function `_derive_key(passphrase, salt, key_len)`

How It Is Implemented

```
python
def _derive_key(passphrase: str, salt: bytes, key_len: int) -> bytes:
    return hashlib.pbkdf2_hmac(
        hash_name  = "sha256",               # underlying hash function
        password   = passphrase.encode("utf-8"),  # str → bytes
        salt       = salt,                    # 16 random bytes
        iterations = 600_000,                # cost factor (NIST 2024 minimum)
        dklen      = key_len,                # 32 for AES-256 and ChaCha20
    )
```

What Data It Operates On

- **Input:** A user passphrase (string) + a random salt (bytes)
- **Output:** A 32-byte derived key (bytes), ready to be handed to AES-GCM or ChaCha20
- **Does NOT touch** the actual file data at all

Requirements for Implementation

Requirement	Detail
Python standard library	`hashlib` module
`os.urandom(16)`	For generating the cryptographically secure salt
Iteration count	600,000 (NIST SP 800-132, 2022 recommendation)
Output length	32 bytes (`dklen=32`) for both AES-256 and ChaCha20
Encoding	Passphrase must be encoded to bytes via `.encode("utf-8")` before hashing

4. Encryption Algorithm 1: AES-256-GCM

What It Is

AES (Advanced Encryption Standard) is a symmetric block cipher standardised by NIST in 2001. **GCM** (Galois/Counter Mode) is the mode of operation that describes how AES is applied to data of

arbitrary length.

Together, **AES-256-GCM** is an **Authenticated Encryption with Associated Data (AEAD)** scheme — it simultaneously provides:

- **Confidentiality** (no one can read the data without the key)
- **Integrity** (any tampering with the ciphertext is detected)
- **Authenticity** (proves the data was encrypted by someone who held the key)

Which Data It Is Applied To

Profile: `structured_text`

This algorithm is selected when the data is:

- Mostly printable ASCII characters (printable ratio ≥ 85%)
- Has moderate Shannon entropy (between 3.5 and 7.0 bits/byte)
- Typical examples: **CSV files, JSON exports, XML documents, plain-text reports, database dumps, prose/email content**

In the sample data: `sample_data/employees.csv` → entropy ≈ 5.1, printable ratio = 100% → profile: `structured_text` → **AES-256-GCM selected**

How AES Works (Block Cipher Core)

AES processes data in **128-bit (16-byte) blocks**. For each block, it applies 14 rounds (for AES-256) of four transformations:

```
Round Key Addition  →  SubBytes (S-box lookup)  →  ShiftRows  →  MixColumns
     ↑_____________________________________|
            (repeated 14 times for AES-256)
```

- **SubBytes:** Each byte is replaced by a fixed non-linear lookup table (S-box). Provides confusion.
- **ShiftRows:** Rows of the 4×4 byte matrix are cyclically shifted. Provides diffusion.
- **MixColumns:** Columns are multiplied in Galois Field GF(2^8). Further diffusion.
- **AddRoundKey:** Each byte is XORed with a byte of the round key (derived from the 256-bit master key).

How GCM Works (Mode of Operation)

GCM turns AES (a block cipher) into a stream cipher using a **counter**:

```
Key (32 bytes) + Nonce (12 bytes)
         │
         ▼
AES-CTR: generates a keystream block for each counter value
         │
         ▼  XOR
Plaintext block 1  →  Ciphertext block 1
Plaintext block 2  →  Ciphertext block 2
...
         │
         ▼
GHASH (Galois Hash): authenticates all ciphertext blocks
         │
         ▼
16-byte Authentication Tag (appended to ciphertext)
```

Because GCM uses CTR mode, **no padding is needed —** the ciphertext is exactly the same length as the plaintext, plus 16 bytes for the authentication tag.

The Nonce

The nonce ("Number Used Once") is **12 random bytes** generated with `os.urandom(12)`. The same (key, nonce) pair must NEVER be reused — reuse breaks GCM's security completely. A fresh nonce is generated for every single encryption call in this project.

Where It Is Implemented

File:

- Encryption: `_aes_gcm_encrypt(data, passphrase)` — lines 56–90
- Decryption: `_aes_gcm_decrypt(container, passphrase)` — lines 93–122

How It Is Implemented (Step by Step)

Encryption:

```
python
def _aes_gcm_encrypt(data: bytes, passphrase: str) -> dict:
    from cryptography.hazmat.primitives.ciphers.aead import AESGCM

    salt   = os.urandom(16)      # Step 1: random 128-bit salt
    nonce  = os.urandom(12)      # Step 2: random 96-bit nonce
    key    = _derive_key(passphrase, salt, 32)  # Step 3: PBKDF2 → 32-byte key

    aesgcm     = AESGCM(key)     # Step 4: initialise AES-GCM cipher
    ciphertext = aesgcm.encrypt(nonce, data, None)  # Step 5: encrypt + tag

    # ciphertext = encrypted_data + 16-byte GCM authentication tag
    return {"algorithm": "AES-256-GCM", "salt": salt,
            "nonce": nonce, "ciphertext": ciphertext}
```

Decryption:

```
python
def _aes_gcm_decrypt(container: dict, passphrase: str) -> bytes:
    from cryptography.hazmat.primitives.ciphers.aead import AESGCM
    from cryptography.exceptions import InvalidTag

    # Step 1: Reproduce the exact same key using the stored salt
    key    = _derive_key(passphrase, container["salt"], 32)
    aesgcm = AESGCM(key)

    try:
        # Step 2: Verify authentication tag, THEN decrypt
        plaintext = aesgcm.decrypt(container["nonce"], container["ciphertext"],
None)
    except InvalidTag:
        # Wrong passphrase or tampered ciphertext — both detected here
        raise ValueError("Decryption failed — wrong passphrase or corrupted data")

    return plaintext
```

Requirements for Implementation

Requirement	Detail
Third-party library	`cryptography` (`pip install cryptography`)
Key size	256 bits = 32 bytes
Nonce size	96 bits = 12 bytes (NIST SP 800-38D recommendation)
Salt size	128 bits = 16 bytes
Authentication tag	16 bytes (128 bits), appended to ciphertext automatically
Key derivation	PBKDF2-HMAC-SHA256 with 600,000 iterations
User input required	Passphrase (any length, any characters)
Mode of operation	GCM (no padding, stream-like, authenticated)
NIST standard	FIPS 197 (AES), SP 800-38D (GCM)

5. Encryption Algorithm 2: ChaCha20-Poly1305

What It Is

ChaCha20-Poly1305 is a modern AEAD (Authenticated Encryption with Associated Data) stream cipher designed by Daniel J. Bernstein ("djb") in 2008. It is standardised in **RFC 8439** and is used in TLS 1.3, WireGuard VPN, and Android/iOS secure messaging.

Like AES-256-GCM, it provides confidentiality + integrity + authenticity — but via a fundamentally different internal design.

Which Data It Is Applied To

Profile: `high_entropy_binary` and `binary_stream`

This algorithm is selected when the data is:

- High Shannon entropy (> 7.0 bits/byte) — already-compressed, already-encrypted, or multimedia data
- Binary content with low printable ratio — executable files, archive files, database binary dumps

- Arbitrary-length streams where block boundaries don't apply

Typical examples: `.zip` / `.gz` compressed files, JPEG/PNG/MP4 media files, SQLite database binaries, firmware images, already-encrypted blobs

How ChaCha20 Works

ChaCha20 is a **stream cipher —** it generates a keystream (a sequence of pseudo-random bytes) and XORs it with the plaintext. The core operation is the **quarter-round function** applied to a 4×4 matrix of 32-bit words:

```
State matrix (512 bits = 64 bytes):
┌──────────┬──────────┬──────────┬──────────┐
│ "expa"   │ "nd 3"   │ "2-by"   │ "te k"   │  ← constants
├──────────┼──────────┼──────────┼──────────┤
│ key[0]   │ key[1]   │ key[2]   │ key[3]   │  ← 256-bit key (words 4–7)
├──────────┼──────────┼──────────┼──────────┤
│ key[4]   │ key[5]   │ key[6]   │ key[7]   │  ← 256-bit key (words 8–11)
├──────────┼──────────┼──────────┼──────────┤
│ counter  │ nonce[0] │ nonce[1] │ nonce[2] │  ← 32-bit counter + 96-bit nonce
└──────────┴──────────┴──────────┴──────────┘

20 rounds of quarter-round operations (add-rotate-XOR on pairs of words)
→ 64-byte keystream block

Repeat with counter incremented for next 64-byte plaintext block
→ XOR keystream with plaintext → ciphertext
```

The quarter-round uses only **addition, rotation (bit-shift), and XOR (ARX)**. There are no lookup tables or S-boxes, making ChaCha20 inherently immune to cache-timing side-channel attacks.

How Poly1305 Works

Poly1305 is a **one-time message authentication code (MAC)** that authenticates the ciphertext:

```
ciphertext  →  Poly1305(key derived from ChaCha20 block 0)
                        │
                        ▼
              16-byte authentication tag
```

The tag is appended to the ciphertext. On decryption, Poly1305 recomputes the expected tag — if it doesn't match, the ciphertext was tampered with and `InvalidTag` is raised before any plaintext is returned.

AES-GCM vs ChaCha20-Poly1305

Property	AES-256-GCM	ChaCha20-Poly1305
Type	Block cipher (CTR mode)	Stream cipher
Performance without AES-NI hardware	Slow	**Fast**
Performance with AES-NI hardware	**Fast**	Comparable
Padding required	No (CTR)	No (stream)
Side-channel resistance	Depends on hardware	Strong (ARX design)
Authentication tag	16 bytes (GHASH)	16 bytes (Poly1305)
Best for	Structured text, small files	Binary streams, mobile/ARM

Where It Is Implemented

File:

- Encryption: `_chacha20_encrypt(data, passphrase)` — lines 129–163
- Decryption: `_chacha20_decrypt(container, passphrase)` — lines 166–195

How It Is Implemented (Step by Step)

Encryption:

```
python
def _chacha20_encrypt(data: bytes, passphrase: str) -> dict:
    from cryptography.hazmat.primitives.ciphers.aead import ChaCha20Poly1305

    salt   = os.urandom(16)      # Step 1: random 128-bit salt
    nonce  = os.urandom(12)      # Step 2: random 96-bit nonce
    key    = _derive_key(passphrase, salt, 32)  # Step 3: PBKDF2 → 32-byte key

    chacha     = ChaCha20Poly1305(key)  # Step 4: initialise cipher
    ciphertext = chacha.encrypt(nonce, data, None)  # Step 5: encrypt + Poly1305
tag

    return {"algorithm": "ChaCha20-Poly1305", "salt": salt,
            "nonce": nonce, "ciphertext": ciphertext}
```

Decryption:

```
python
def _chacha20_decrypt(container: dict, passphrase: str) -> bytes:
    from cryptography.hazmat.primitives.ciphers.aead import ChaCha20Poly1305
    from cryptography.exceptions import InvalidTag

    key    = _derive_key(passphrase, container["salt"], 32)
    chacha = ChaCha20Poly1305(key)

    try:
        plaintext = chacha.decrypt(container["nonce"], container["ciphertext"],
None)
    except InvalidTag:
        raise ValueError("Decryption failed — Poly1305 tag mismatch")

    return plaintext
```

Requirements for Implementation

Requirement	Detail
Third-party library	`cryptography` (`pip install cryptography`)
Key size	256 bits = 32 bytes
Nonce size	96 bits = 12 bytes (as required by the `cryptography` library)
Salt size	128 bits = 16 bytes
Authentication tag	16 bytes (Poly1305), appended to ciphertext automatically
Key derivation	PBKDF2-HMAC-SHA256 with 600,000 iterations
User input required	Passphrase (any length, any characters)
RFC standard	RFC 8439
No padding needed	Stream cipher — works on any byte length

6. Encryption Algorithm 3: Multi-byte XOR

What It Is

XOR (Exclusive OR) is a **bitwise logical operation** and the conceptual foundation of all stream ciphers. In this project it is implemented as a **single-byte repeating key XOR**, where every byte of the plaintext is XORed with the same key byte.

> ⚠️ **This is an Educational / Playground algorithm. It is NOT secure for real data.** It is included to teach and demonstrate how stream ciphers work at their most basic level.

The XOR Operation

XOR outputs 1 when inputs differ, 0 when they match:

```
0 XOR 0 = 0
0 XOR 1 = 1
1 XOR 0 = 1
1 XOR 1 = 0
```

The crucial property: **XOR is its own inverse**:

```
plaintext  XOR key = ciphertext
ciphertext XOR key = plaintext
```

This means the **same function** performs both encryption and decryption.

Which Data It Is Applied To

Profile: Any — the user must manually select this algorithm via the override menu. (It is not automatically recommended for any profile, since it provides no real security.)

Appropriate for: Demonstration, teaching, or obfuscation of non-sensitive data. **Not appropriate for:** Any data where confidentiality actually matters.

How It Is Implemented (Step by Step)

```
python
def _xor_encrypt(data: bytes, key_byte: int) -> dict:
    # Validate key is a single byte (0 to 255)
    if not 0 <= key_byte <= 255:
        raise ValueError("XOR key_byte must be an integer in [0, 255].")

    # XOR every byte of data with key_byte using a generator expression
    # bytes(generator) materialises the results into an immutable bytes object
    ciphertext = bytes(b ^ key_byte for b in data)

    return {
        "algorithm" : "Multi-byte XOR",
        "key_byte"  : key_byte,
        "ciphertext": ciphertext,
    }

def _xor_decrypt(container: dict, key_byte: int) -> bytes:
    # XOR is symmetric — decryption is identical to encryption
    result = _xor_encrypt(container["ciphertext"], key_byte)
    return result["ciphertext"]
```

Worked Example

```
Plaintext:    H    e    l    l    o
ASCII values: 72   101  108  108  111
Key byte:     42   42   42   42   42   (key = 42 = 0x2A, repeating)
              —    —    —    —    —
XOR result:   98   127  70   70   69
              —    —    —    —    —
Ciphertext:   b    DEL  F    F    E

Decryption (XOR again with same key):
Ciphertext:   98   127  70   70   69
Key byte:     42   42   42   42   42
XOR result:   72   101  108  108  111  ← original "Hello" restored ✔
```

Key Space and Why It Is Insecure

The key is a single byte: values 0 through 255. That is **only 256 possible keys —** an attacker can try every one of them in microseconds. For each trial key, the attacker XORs the ciphertext and checks if the result is readable text.

Special case — key_byte = 0: XOR with 0 is the identity function: `b XOR 0 = b`. The "ciphertext" is identical to the plaintext, providing zero protection.

Requirements for Implementation

Requirement	Detail
Python standard library	Built-in `bytes` and integer XOR operator (`^`)
Key	A single integer in range [0, 255] — `int(params["xor_key_byte"])`
No library needed	Pure Python — no `cryptography` package required
User input required	A key byte (integer 0–255), entered at the CLI prompt
Validation	Must check `0 <= key_byte <= 255` before use
Works on	Any `bytes` input — text or binary

7. Encryption Algorithm 4: ROT Cipher (Caesar Cipher)

What It Is

The ROT (Rotate) Cipher is a **substitution cipher** — it replaces each letter of the plaintext with a different letter a fixed number of positions further along in the alphabet. It is a generalisation of the **Caesar Cipher** (named after Julius Caesar, who used a shift of 3).

ROT-13 (shift = 13) is the most famous variant: applied twice, it returns the original text, because 13+13 = 26 = one full alphabet cycle.

> ⚠️ **This is an Educational / Playground algorithm. It is NOT secure for real data.** There are only 25 possible keys (shifts 1–25). Any attacker breaks it immediately.

Which Data It Is Applied To

Profile: `classical_cipher_text`

This is selected when data has:

- Very high printable ratio (≥ 90%)
- Very low Shannon entropy (< 3.5 bits/byte)

This profile represents **simple, highly repetitive human-readable text —** short prose, repeated phrases, or data that was already protected by a similar classical cipher.

Typical examples: Simple text files, repeated phrases, short messages, text that mimics what would have been sent via historical ciphers.

How It Works

Only **alphabetic characters** are shifted. Digits, punctuation, spaces, and all other characters pass through unchanged.

```
Original alphabet:  A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Shifted (shift=3):  D E F G H I J K L M N O P Q R S T U V W X Y Z A B C
```

Mathematical formula for uppercase letters:

```
ciphertext_byte = ( (plaintext_byte − 65 + shift) mod 26 ) + 65
```

Mathematical formula for lowercase letters:

```
ciphertext_byte = ( (plaintext_byte − 97 + shift) mod 26 ) + 97
```

The `mod 26` ensures the shift wraps around the alphabet (e.g. Z shifted by 3 becomes C).

Decryption formula: Shift in the reverse direction:

```
reverse_shift = (26 − shift) mod 26
```

For shift = 13: reverse = (26 − 13) mod 26 = 13. That is why ROT-13 is self-inverse.

How It Is Implemented (Step by Step)

```
python
def _rot_encrypt(data: bytes, shift: int) -> dict:
    # Validate shift is in range [1, 25]
    if not 1 <= shift <= 25:
        raise ValueError("ROT shift must be an integer in [1, 25].")

    # Inner helper function (closure) — applies the shift to a single byte
    def _shift_byte(b: int, s: int) -> int:
        if ord('A') <= b <= ord('Z'):        # Uppercase letter
            return (b - ord('A') + s) % 26 + ord('A')
        if ord('a') <= b <= ord('z'):        # Lowercase letter
            return (b - ord('a') + s) % 26 + ord('a')
        return b                             # Non-alphabetic: unchanged

    # Apply _shift_byte to every byte and collect into a new bytes object
    ciphertext = bytes(_shift_byte(b, shift) for b in data)

    return {
        "algorithm" : "ROT Cipher",
        "shift"     : shift,
        "ciphertext": ciphertext,
    }

def _rot_decrypt(container: dict, shift: int) -> bytes:
    # Compute the reverse shift
    reverse_shift = (26 - shift) % 26
    # Handle edge case: shift=26 would give reverse_shift=0 which breaks the
encrypt guard
    result = _rot_encrypt(container["ciphertext"], reverse_shift if reverse_shift
else 26)
    return result["ciphertext"]
```

Worked Example (ROT-13)

```
Plaintext:   H   e   l   l   o   ,     W   o   r   l   d
Shift = 13
──────────────────────────────────────────────────────────
U   r   y   y   b   ,     J   b   e   y   q

Decryption (shift = 13 again, since ROT-13 is self-inverse):
U   r   y   y   b   ,     J   b   e   y   q
──────────────────────────────────────────────────────────
H   e   l   l   o   ,     W   o   r   l   d   ← restored ✔
```

Why It Is Insecure

- **Key space = 25 values** (shifts 1 through 25). An attacker tries all 25 in one second.
- **Frequency analysis:** The letter 'E' is the most common in English. If a particular ciphertext letter appears most often, it likely encodes 'E', revealing the shift instantly.
- **No authentication:** Any modification to the ciphertext produces a different (garbled) plaintext without any error — there is no integrity check.

Requirements for Implementation

Requirement	Detail
Python standard library	Built-in `bytes`, `ord()`, modulo arithmetic (`%`)
No library needed	Pure Python — no third-party packages required
Key	An integer shift in [1, 25], entered at the CLI prompt
User input required	The shift integer
Validation	Must check `1 <= shift <= 25` before use
Works on	ASCII / UTF-8 text bytes. Non-alphabetic bytes are passed through unchanged. Binary data is technically accepted but the shift only affects bytes in the ranges 65–90 (A–Z) and 97–122 (a–z).

8. Algorithm Comparison Table

Property	AES-256-GCM	ChaCha20-Poly1305	Multi-byte XOR	ROT Cipher
Category	AEAD Block Cipher	AEAD Stream Cipher	Symmetric XOR	Substitution Cipher
Tier	Production	Production	Educational	Educational
Cipher type	Block (128-bit blocks)	Stream	Stream (trivial)	Substitution
Key size	256 bits (32 bytes)	256 bits (32 bytes)	8 bits (1 byte)	5 bits (shift 1–25)
Effective key space	2^{256}	2^{256}	$2^8 = 256$ values	25 values
Authenticated?	✅ Yes (GCM tag)	✅ Yes (Poly1305)	❌ No	❌ No
Padding needed?	No (GCM = CTR mode)	No (stream cipher)	No	No
Nonce required?	✅ 12 bytes	✅ 12 bytes	❌ No	❌ No
Salt required?	✅ 16 bytes (PBKDF2)	✅ 16 bytes (PBKDF2)	❌ No	❌ No
Key derivation	PBKDF2-HMAC-SHA256	PBKDF2-HMAC-SHA256	None (direct)	None (direct)
User input	Passphrase	Passphrase	Key byte (0–255)	Shift integer (1–25)
Detects tampering?	✅ Yes	✅ Yes	❌ No	❌ No
Third-party library	`cryptography`	`cryptography`	None needed	None needed
Recommended for	CSV, JSON, XML, text	Binary, compressed, media	Teaching only	Teaching only
Data profile triggered	`structured_text`	`high_entropy_binary`, `binary_stream`	Manual override only	`classical_cipher_text`
Overhead per file	+16 bytes tag + 28 bytes nonce/salt	+16 bytes tag + 28 bytes nonce/salt	0 bytes	0 bytes

Property	AES-256-GCM	ChaCha20-Poly1305	Multi-byte XOR	ROT Cipher
NIST / RFC standard	FIPS 197, SP 800-38D	RFC 8439	None	None
Brute-force resistance	Effectively infinite	Effectively infinite	256 guesses	25 guesses

Summary

This project implements **six algorithms in total —** two are analysis algorithms and four are encryption algorithms:

#	Algorithm	Category	File	Purpose
1	Shannon Entropy	Analysis	`profiler.py`	Measure data randomness to choose cipher
2	PBKDF2-HMAC-SHA256	Key Derivation	`crypto_engine.py`	Convert passphrase into a strong cryptographic key
3	AES-256-GCM	Encryption (Production)	`crypto_engine.py`	Secure encryption of structured text data
4	ChaCha20-Poly1305	Encryption (Production)	`crypto_engine.py`	Secure encryption of binary / high-entropy streams
5	Multi-byte XOR	Encryption (Educational)	`crypto_engine.py`	Demonstrate stream cipher fundamentals
6	ROT Cipher	Encryption (Educational)	`crypto_engine.py`	Demonstrate substitution cipher fundamentals

Algorithms 1 and 2 operate silently in the background to support the encryption algorithms. Algorithms 3 and 4 are the production-grade, cryptographically secure choices. Algorithms 5 and 6 exist to make the system useful as a **teaching and demonstration tool**.